

Unit 4: OOPS Concept

Object Oriented Concept

- All languages those supports the concepts of OOP they are considered as **OOP programming language**.
- All OOP based languages supports the following concepts.
 - Class and Object
 - Abstraction and Encapsulation
 - Constructor and Destructor
 - Inheritance
 - Polymorphism
- **Class**
 - Class is a group of **methods** and **variables** where methods are known as **Member function** and variables are known as **Data Member**.
 - In vb.net class is created using **Class Keyword** and completed with **End Class**.
 - Once class is created you can created any number of objects.

Syntax:

```
[accessmodifier][Shadows][MustInherit|NotInheritable]
Class <Classname>
    [Inherits classname]
    [Implements interfacednames]
    [statements]
End Class
```

where

- **Accessmodifier** defines the access levels of the class; it has values as - Public, Protected, Friend, Protected Friend and Private. Optional.
- **Shadows** indicate that the variable re-declares and hides an identically named element, or set of overloaded elements, in a base class. Optional.
- **MustInherit** specifies that the class can be used only as a base class and that you cannot create an object directly from it, i.e., an **abstract class**. Optional.
- **NotInheritable** specifies that the class cannot be used as a base class.
- **Inherits** specifies the base class it is inheriting from.
- **Implements** specify the interfaces the class is inheriting from.

Example

- Add new class file to the project and give proper name (Class1.vb). The extension of the class is **.vb**
- Write down the following code in Class1.vb File

Unit 4: OOPS Concept

```
Public Class Class1 'class name
    Dim a, b As Integer
    'Class Method
    Sub getdata(ByVal i As Integer, ByVal j As Integer)
        a = i
        b = j
    EndSub
    Sub disp() 'class method
        MsgBox("a==" & a & "b==" & b)
    EndSub
    Function sum() As Integer 'function
        Return a + b
    EndFunction
End Class
```

- Add windows form name frmclass.vb and write down the following code in btnclass click event.

```
Private Sub btnclass_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnclass.Click
    Dim obj As New Class1 'object is created
    obj.getdata(10, 20) 'method is called
    obj.disp()
    MsgBox("the sum of two no is " & obj.sum())
End Sub
```

Object

- It is runtime entity.
- It is member of class which is able to access methods and variables of class which has been declared in Public Mode.
- In vb.net ,it can be created as follows:

Syntax:

- | |
|--|
| <ul style="list-style-type: none">○ <object> As New <Class Name> |
|--|

Unit 4: OOPS Concept

- Example:

```
PrivateSub btnclass_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btnclass.Click
Dim obj AsNewClass1 'object is created
obj.getdata(10, 20) 'method is called
obj.disp()
MsgBox("the sum of two no is "& obj.sum())
```

EndSub

- **Abstraction**

- It refers to the act of representing essentials features without including the background details or explanations.
- It defines way to abstract or hide your data and members from outside class.
- Classes use the concept of Abstraction.
- When we define a classes then different accessibility mode define the access level of variables and methods. They are **Public, Private, Protected**.

Example:

```
PublicClassClass1'class name
Dim a, b AsInteger
'Class Method
Sub getdata(ByVal i AsInteger, ByVal j AsInteger)
a = i
b = j
EndSub
Sub disp() 'class method
MsgBox("a=="& a &"b=="& b)
EndSub
Function sum() AsInteger'function
Return a + b
EndFunction
EndClass
```

- **Encapsulation**

- It is a process to bind data and methods in a unit called class. When we define class that process itself define the concept of Encapsulation.
- It can protect your data from accidental corruption.
- Rather than defining the data in the form of public, we can declare those fields as private.

Unit 4: OOPS Concept

- **Example**

```
PublicClassClass1'class name
Dim a, b AsInteger
'Class Method
Sub getdata(ByVal i AsInteger, ByVal j AsInteger)
    a = i
    b = j
EndSub
Sub disp() 'class method
    MsgBox("a==" & a & "b==" & b)
EndSub
Function sum() AsInteger'function
    Return a + b
EndFunction
EndClass
```

- **Constructor**

- It is a special member function which is used to initialize the object of its class.
- It is automatically called when object is created.
- In vb.net, The name of constructor is **NEW()**
- It can be **overloaded**.
- There are 2 types of constructor such as constructor with **arguments** and **without arguments**.

- **Example**

Add Class file and give name ClsConstuctor

```
PublicClassClsConstuctor
Dim i AsInteger
Dim n AsInteger
'Constructor without arguments
SubNew()
    MsgBox("I am Default Constuctor")
EndSub
'Constructor with arguments
SubNew(ByVal i AsInteger)
    n = i
    MsgBox("I am paramiterized Constuctor")
    MsgBox(n)
EndSub
ProtectedOverridesSub finalize()
    n = 0
EndSub
```

Unit 4: OOPS Concept

```
MsgBox("I m Desctuctor")  
EndSub  
EndClass
```

Add windows form and write down the following code in btnconstructor click event.

```
PrivateSub btnConstructor_Click(ByVal sender As System.Object, ByVal e  
As System.EventArgs) Handles btnConstructor.Click  
Dim x As New ClsConstructor  
Dim y As New ClsConstructor(10)  
EndSub
```

- **How to call Base Class Constructor (imp)**
 - It can be called using **MyBase** keyword.
 - It cannot be used to access private members of class.
 - It refers to the immediate base class and its inherited members
- **Example:**

Unit 4: OOPS Concept

```
PublicClassClsBase
Private x, y AsInteger
SubNew()
    x = 0
    y = 0
EndSub
SubNew (ByVal a AsInteger, ByVal b AsInteger)
    x = a
    y = b
EndSub
EndClass

PublicClassclsderive
InheritsClsBase
Private z AsInteger
SubNew()
    MyBase.New()
    z = 0
EndSub
SubNew(ByVal a AsInteger, ByVal b AsInteger, ByVal c AsInteger)
    MyBase.New(a, b)
    z = c
    MsgBox(a & b & c)
EndSub
EndClass
```

Unit 4: OOPS Concept

Add windows form and write down the following code in **btnconstructor** click event.

```
PrivateSub btnConstructor_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnConstructor.Click
    Dim obj As New clsderive
    Dim obj1 As New clsderive(4, 6, 7)
EndSub
```

- **Destructor**

- It is a special function which is called automatically when a class is destroyed.
- A destructor is also known as finalizer. It is the last method run by class
- Within a destructor we can place code to clean up the object after it is used ,which might include decrement counters releasing resources
- It cannot be overloaded.

- **Example:**

```
Protected Overrides Sub finalize()
    n = 0
    MsgBox("I m Desctuctor")
EndSub
```

- **Inheritance**

- It is a process of creating a new class called derives class from the existing class called Base Class.
- The existing class is known as parent, base, super class.
- The new class is known as child, derive, sub class.
- It is also known as code reusability.
- It can be created by using inherits keyword.
- A derive class obtain all of the methods, properties, and events of the base class.
- There are 3 modifier related with inheritance.
 - Inherits
 - NotInheritable
 - MustInherit(Abstract Class)

- **Syntax**

Class <Class Name> [inherits] Base Class Name Statements End Class

- **Example**

Add class file name clsbase and write down the following code

Unit 4: OOPS Concept

```
PublicClassClsBase 'Base Class
Protected a AsInteger = 40
Sub disp()
    MsgBox("I am Base Class")
EndSub
EndClass
```

```
PublicClassclsderive 'Derive Class
InheritsClsBase
Dim b AsInteger = 20
Sub show()
    MsgBox("value from Base Class=" & a)
    MsgBox("value from Derive Class=" & b)
    MsgBox("I am Derived Class Method")
EndSub
EndClass
```

Add windows form and write down the following code in btninheritance click event

```
PrivateSub btninheritance_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btninheritance.Click
Dim x AsNewclsderive
    x.disp() 'Base Class Method
    x.show() 'Base + derive Class'
EndSub
```

- **NotInheritable**
 - It is known as seal class. If we write NotInheritable keyword in class then no other class can be derived from this class.
 - It gives surety that your class is not become parent of any class.
- **Example**

Create class clsA as NotInheritable and try to derive it in clsB

```
PublicNotInheritableClassclsA

EndClass
```

```
PublicClassClsB
InheritsclsA 'ClsB cannot inherit from class ClsA because 'ClsA is declared as
NotInheritable

EndClass
```


Unit 4: OOPS Concept

- **MustInherit (Abstract Class)**

- MustInherit Keyword is used to create Abstract Class.
- It specifies that class is intended as a base class.
- It provides Skelton to the derived class. it has no actual code.
- It is not used to create an object.
- If our class contains at least one MustOverride method then our class should be declare as MustInherit class.
- Methods written in Abstract class as MustOverride are automatically appear in the derived class.
- Mustoverride methods must be declared in MustInherit Class.
- No other statements are allowed and especially there is No End Sub or End Function.

- Example

Add Class file and give name clsAbs1

```
PublicMustInheritClassclsAbs1
MustOverrideSub add(ByVal a AsInteger, ByVal b AsInteger)
MustOverrideSub div(ByVal a AsInteger, ByVal b AsInteger)
EndClass
```

Add another Class file and give name clstemp

```
PublicClassclstemp
InheritsclsAbs1

PublicOverridesSub add(ByVal a AsInteger, ByVal b AsInteger)
    MsgBox(a + b)

EndSub

PublicOverridesSub div(ByVal a AsInteger, ByVal b AsInteger)
    MsgBox(a / b)

EndSub
EndClass
```

Add windows form and write down the following code in btnabstactclass_Click event

```
PrivateSub btnabstactclass_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles btnabstactclass.Click
Dim obj AsNewclstemp
```

Unit 4: OOPS Concept

```
obj.add(10, 20)
obj.div(20, 10)
```

EndSub

- **Polymorphism**

- It means “one name, multiple forms
- There are two types of it **(1) Design time (2)Run time**
- **Design time** polymorphism is achieved by **function overloading**
- **Run time** polymorphism is achieved by **function overriding**
- Function overloading means we can declare more than one functions with the **same name but arguments are different.**

Example

Add Class file and write this code (it is the example of function overloading)

PublicClassClsmath

```
Function add(ByVal a AsInteger)
Return MsgBox(a)
EndFunction
```

```
Function add(ByVal a AsInteger, ByVal b AsInteger)
Return MsgBox(a + b)
EndFunction
```

```
Function add(ByVal a AsInteger, ByVal b AsInteger, ByVal c AsInteger)
Return MsgBox(a + b + c)
EndFunction
EndClass
```

Write this code on btnpoly_click event

```
PrivateSub btnpoly_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnpoly.Click
Dim obj AsNewClsmath
obj.add(5)
obj.add(5, 5)
obj.add(10, 20, 30)
EndSub
```

Unit 4: OOPS Concept

Run time polymorphism (function overriding)

- Function overriding means when we declare more than one function with same name **but in different class** and the relationship between class is **parent -child**
- The overriding concept achieve only in inheritance.
- Derive class inherits methods from its base class
- If an inherited property or method needs to behave differently in the derived class.
- **Overridable keyword** is used to mark a function as override. We can redefine overridable methods in derived class
- **Theoverrides keyword** is used to mark that a function is overriding some base class function.
- The **overrides** keyword overrides an overridable property or method defined in the base class

Example

```
PublicClass Clsoverridable
PublicClass base
OverridableSub disp()
    MsgBox("Base class")
EndSub
EndClass
PublicClass clsderived
Inherits base
PublicOverridesSub disp()
    MyBase.disp()
    MsgBox("Derived Class")
EndSub
EndClass
EndClass
```

Write this code in btnoverriding_click Event

```
PrivateSub btnoverriding_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles btnoverriding.Click
Dim obj As New Clsoverridable.clsderived
obj.disp()
EndSub
```

Unit 4: OOPS Concept

- **Interface**

Why interface used?

We cannot inherit more than one class in vb.net. If we want to inherit then? Use interface

Interface

- Interfaces like classes define a set of properties, methods and events. But unlike classes, an interface does not contain any implementation code
- Interface represents **“Has a” relationship**
- Classes which implement interface write the coding in the interface's method
- We can implement multiple interface
- If we want to use interface then we need to write **“Implements”** keyword

Syntax

```
Public Interface iface1
    ....logic
End Interface
```

Example

Add interface in your project

```
PublicInterfaceInterface1
Sub hi()
EndInterface
```

Add another interface in your project

```
PublicInterfaceInterface2
Sub hello()
EndInterface
```

Add Class File in your project

```
PublicClassClsInterface
ImplementsInterface1, Interface2

PublicSub hi() ImplementsInterface1.hi
    MsgBox("Hi i m interface 1")
EndSub

PublicSub hello() ImplementsInterface2.hello
    MsgBox("Hello i m interface 2")
EndSub
EndClass
```

Unit 4: OOPS Concept

Write down this code in btninterface_click Event

```
PrivateSub btninterface_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles btninterface.Click
    Dim obj As New ClssInterface
        obj.hi()
        obj.hello()
EndSub
```